FIAP

GRADUAÇÃO

45697056

ES
Desenvolvimento Mobile

Prof. Adeilton Meneses

profadeilton.meneses@fiap.com.br

React Native

45697056

Integração com API

Interfaces

45697056

- Definição
  - contrato que especifica métodos e propriedades que uma classe deve implementar
- Sem implementação: apenas a assinatura dos métodos
- Usada para garantir consistência entre diferentes partes do sistema
- Separação de responsabilidades
- Reutilização de código
- Facilita testes e manutenção

```typescript
interface Pessoa {
  nome: string;
  idade: number;
  falar(): void;
}

const joao: Pessoa = {
  nome: "João",
  idade: 30,
  falar() {
    console.log(`Olá, meu nome é ${this.nome}`);
  }
};
```

Documentação - Swagger

45697056

O que é Swagger?

- Conjunto de ferramentas open-source para **documentar, testar e visualizar APIs REST**

Principais Ferramentas Swagger

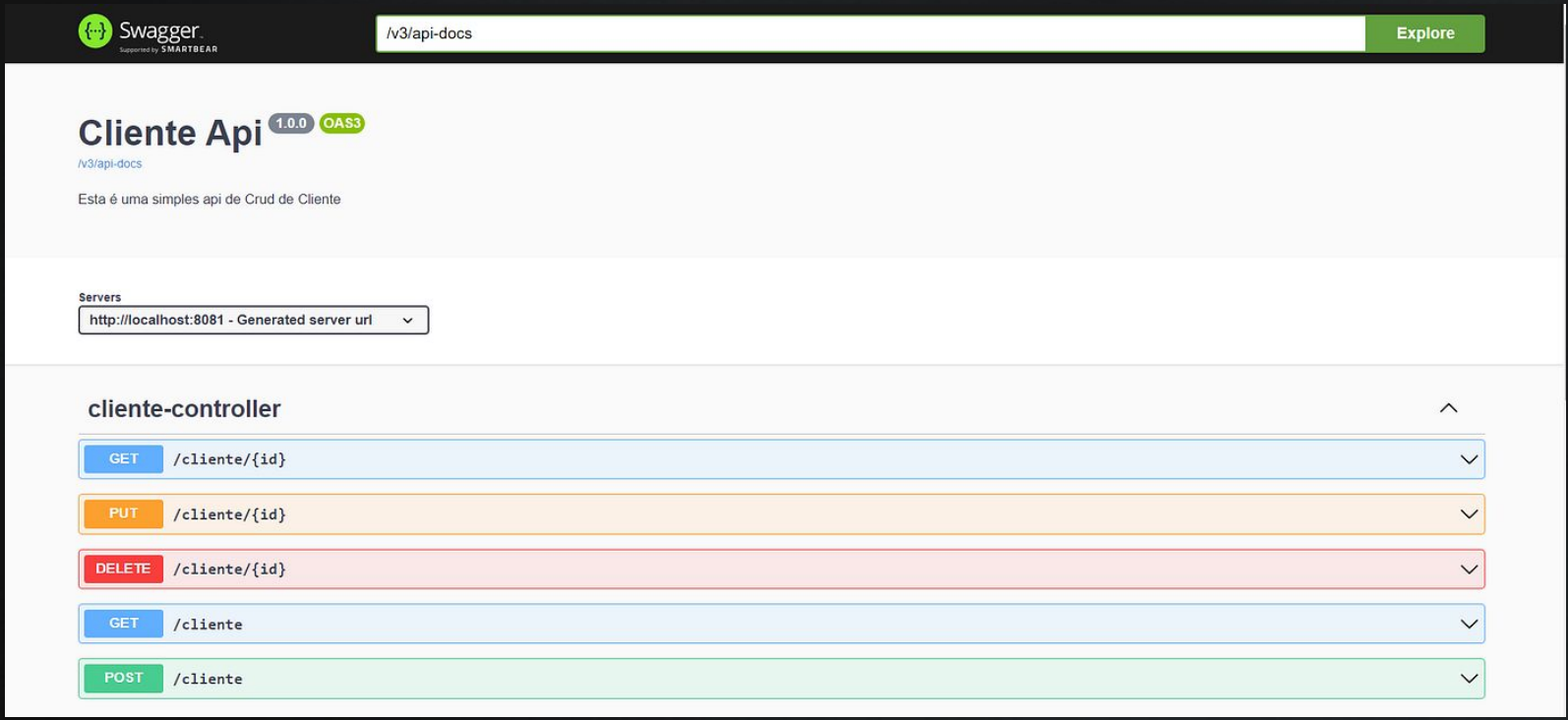
- **Swagger UI**: Interface web para visualizar e testar endpoints
- **Swagger Editor**: Editor online para criar e validar especificações OpenAPI
- **Swagger Codegen**: Gera código cliente/servidor a partir da especificação YAML ou JSON2
- **Swagger Hub**: Plataforma colaborativa para gerenciar APIs

Por que usar Swagger?

- Facilita a comunicação entre equipes de frontend e backend
- Reduz erros de integração
- Permite testes rápidos via interface interativa
- Gera documentação atualizada automaticamente

Documentação

45697056



Documentação - Thunder Client

45697056

GET https://www.receitaws.com.br/v1/cnpj/11319526000155 Send

Query | Headers 2 | Auth | Body | Tests | Pre Run

Query Parameters

parameter	value

Status: 200 OK Size: 2.24 KB Time: 183 ms

Response | Headers 16 | Cookies | Results | Docs


```
{
  "abertura": "10/11/2009",
  "situacao": "ATIVA",
  "tipo": "MATRIZ",
  "nome": "VSTP EDUCACAO S.A.",
  "fantasia": "VSTP EDUCACAO",
  "porte": "DEMAIS",
  "natureza_juridica": "205-4 - Sociedade Anônima Fechada",
  "atividade_principal": [
    {
      "code": "85.31-7-00",
      "text": "Educação superior - graduação"
    }
  ],
  "atividades_secundarias": [
    {
      "code": "85.13-9-00",
      "text": "Ensino fundamental"
    },
    {
      "code": "85.20-1-00",
      "text": "Ensino médio"
    },
    {
      "code": "85.33-3-00",
      "text": "Educação superior - pós-graduação e extensão"
    },
    {
      "code": "85.41-4-00",
```


Response | Chart

Axios

45697056

- O que é Axios?
- Por que usar Axios em vez de ***fetch***?

Recurso	Axios	Fetch API
Parsing de JSON	Automático	Manual (`response.json()`)
Tratamento de erros	Mais intuitivo (captura status HTTP)	Apenas erros de rede são capturados
Timeout de requisição	Suporte nativo (`timeout` config)	Requer `AbortController`
Interceptadores	Sim, para requisições e respostas	Não disponível nativamente
Cancelamento de requisição	Integrado	Requer configuração manual
Transformação de dados	Automática	Manual
Suporte a upload/download progress	Sim	Não nativo

Com Axios:

Js

```js
axios.post('/api/dados', { nome: 'João' })
  .then(res => console.log(res.data))
  .catch(err => console.error('Erro:', err));
```

Com Fetch:

Js

```js
fetch('/api/dados', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ nome: 'João' })
})
  .then(res => {
    if (!res.ok) throw new Error('Erro na resposta');
    return res.json();
  })
  .then(data => console.log(data))
  .catch(err => console.error('Erro:', err));
```

Axios

45697056

- Instalação
  - npm install axios
- Tipos de requisições
  - GET
  - POST
  - PUT
  - DELETE
- No React
  - Requisições dentro de `useEffect`
  - Atualização de estado com dados da API

```
useEffect(() => {
  axios.get('/api/data')
    .then(response => setData(response.data))
    .catch(error => console.error(error));
}, []);
```

Expo API Routes

45697056

- Introdução;
- Funcionamento:
  - Estrutura de arquivos (`/api` dentro do projeto)
  - Como as rotas são definidas (baseadas em arquivos)
  - Métodos HTTP suportados (GET, POST, etc.)
  - São definidas por arquivos com o sufixo ***+api.ts*** dentro da pasta ***app/*** do projeto
- Vantagens:
  - Backend leve embutido no projeto frontend
  - Ideal para protótipos e apps fullstack simples
  - Integração com banco de dados ou serviços externos
  - Evita dependência de servidores externos durante o desenvolvimento

Prática API Routes

45697056

- **Criando projeto**
  - **npx create-expo-app --template**
  - selecione Navigation (typescript)
  - defina o nome: expo-api-routes
  - apagar as pastas:
    - **app, components, constants, fonts**
  - criar pasta: **src/app/index.tsx**

Prática API Routes

45697056

- alterar **tsconfig.json** e adicionar ***src*** dentro dos paths
- alterar em **app.json**
  - web: static -> server
- instalar o **Axios** (npm i axios)
- criar função para chamada das rotas e estilização do APP

- Instalar extensão Postman, Thunder Client… no VS code
- Deploy (com Vercel): **https://docs.expo.dev/router/reference/api-routes/**

Obrigado

2025-Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proíbido sem o consentimento formal, por escrito, do Professor Wellington Cidade Silva Tenorio.